

Software Quality Testing of a Personal Expense and Budget Management Web Application

Joint submission report

Team member	Index number	Primary contribution
Erjon Koxha	231509	Backend, ISP, PostgreSQL, property tests, load and security
Bora Alili	231504	Frontend, MSW integration, browser UI, Playwright E2E

Submission date: 28 September 2026

Source code and test repositories

Application and all test code share one public repository:

<https://github.com/erikoxha/expense-manager>

Direct test locations: [backend/tests](#) | [frontend/src/tests](#) | [e2e](#) | [load-tests](#) | [security](#)

Verified outcome

The merged application passed 161 backend tests, 37 frontend component/integration tests, four browser UI tests and one real-app E2E workflow in GitHub Actions. Separate local load and security assessments provide additional evidence and documented limitations.

Both testing contributions were merged after automated review and combined local verification. Final continuous integration results are distinguished from earlier local load and security measurements. The tested source version is linked in the final evidence section.

1. Application and team scope

Ledger is an application built by the team with AI assistance for practical software testing. Users register, sign in, maintain income/expense categories and transactions, define category budgets, and inspect summary and monthly statistics. All private records belong to the authenticated user. The project uses synthetic data throughout its automated tests.

Layer	Implementation and testing boundary
Frontend tech stack	React 19, React Router, Recharts; forms, navigation, validation feedback and financial charts.
Backend tech stack	Django 5.2.17 and Django REST Framework 3.16.1; JWT access/refresh tokens, serializers, ownership and financial aggregation.
Persistence / DB	PostgreSQL 16; real migrations, foreign keys, constraints and ORM operations. SQLite is not substituted.
Runtime environment	Local Docker Compose, Gunicorn with two workers and Nginx. Load/security tests used the local setup; no public cloud deployment was assessed.

Business rules defining expected results

Amounts are positive Decimal(12,2), from 0.01 to 9,999,999,999.99. Extra decimal precision is rejected. Transaction type must match the category; budgets use expense categories. Budget periods include both boundary dates. Remaining budget equals amount minus eligible spending; percentage may exceed 100, while the visual progress bar is capped. Balance equals income minus expenses. Display currency is EUR; conversion is not implemented.

Names are trimmed and case-insensitively unique per user and category type. Referenced categories cannot be deleted or have their type changed. Foreign record identifiers return 404 for private CRUD requests. Refresh tokens rotate and are blacklisted; logout now requires authenticated access and refresh-token ownership. Already-issued access tokens remain valid until their short expiry.

Ownership and collaboration

Erjon owns backend/API/database tests, ISP documentation, Hypothesis, k6, ZAP and related evidence. Bora owns component/form tests, MSW integration, real browser design checks, E2E and frontend coverage. GitHub retains separate contribution histories in [PR #1](#) and [PR #2](#). AI assisted implementation, test preparation, execution and review; the results below refer to actual runs, not assumed outcomes.

Architecture and contracts: [docs/ARCHITECTURE.md](#); [docs/API.md](#); [docs/DESIGN_SYSTEM.md](#).

Terminology and abbreviations

Selected tools, security concepts and coverage measures used in this project are defined below.

Term	Meaning in this project
MSW	Mock Service Worker. It supplies controlled API responses during frontend tests, including failures and delays.
PBKDF2	Password-Based Key Derivation Function 2. A salted, repeated computation used to store and check passwords without storing the original password.
ZAP / DAST	Zed Attack Proxy / Dynamic Application Security Testing. ZAP checks a running application for potential security weaknesses.
CSP / XSS	Content Security Policy / Cross-Site Scripting. CSP restricts browser content; XSS executes unwanted script in a page.
p95	The 95th percentile: 95% of measured request durations are at or below this value.
Statement / branch coverage	The proportion of executable statements and decision outcomes exercised within the configured measurement scope.

2. Strategy and coverage criteria

The testing strategy combines specification-based examples and boundaries with structural coverage and full-stack checks. Each layer has a defined expected result or acceptance measure: API status and persisted state, exact Decimal totals, rendered UI behavior, browser geometry, workload metrics or investigated scanner evidence.

Layer / criterion	Acceptance and measured scope
Backend functional tests	All required collected cases pass. Real Django/DRF and PostgreSQL integrations complement isolated budget arithmetic.
Backend structural coverage	At least 90% statements and 85% branches in users, expenses and config; omit only migrations and package initializers.
Frontend structural coverage	At least 85% statements and 80% branches across src JS/JSX, excluding main.jsx bootstrap and test files. Enforced in Vitest.
ISP	Each Choice Coverage for the explicitly modeled blocks; boundary representatives supplement it. Not all-combinations or pairwise coverage.
Load	HTTP failures below 1%, request p95 below 500 ms, business checks at least 99% for the declared local workload.
Security	Investigate every alert; no unresolved confirmed high-risk finding from the investigated scan. Retain false-positive evidence and residual risks.

Requirements traceability

Requirement identifiers connect a requirement to its tests: AUTH = authentication; OWN = ownership; TX = transactions; CAT = categories; BUD = budgets; STAT = statistics; FILTER = filtering; DB = database; ERR = error handling; PROP = property-based testing; PERF = performance; SEC = security. The mapping table records which tests check each requirement. Exact mappings are in [docs/testing/PLAN.md](#).

Techniques beyond the supplied course outline

API and user-interface testing, mocking, input partitions and structural coverage form the foundation of the strategy. Additional methods include Hypothesis-generated property tests, k6 load testing and authenticated dynamic security scanning with ZAP. These extend the range of inputs and operational conditions examined.

Branch coverage records whether the tests executed the alternative outcomes of code decisions. The measured 100% result applies to the configured backend scope; it does not establish that every possible input or sequence of actions behaves correctly. More demanding criteria, such as checking the independent effect of each condition in a compound decision, were outside the scope.

3. ISP transaction input model

Input Space Partitioning (ISP) divides a selected input domain into behaviorally meaningful blocks. The criterion is Each Choice Coverage: every listed block has an exercised representative. This explicit traceability model was completed after implementation; it is not presented as a detailed model written before the tests.

The valid base request is an authenticated Alice creating an EXPENSE transaction in her own expense category, amount 12.34, date 2026-09-15 and description Food shop. Invalid cases usually vary one field so another error cannot mask the expected outcome.

Amount block	Representative	Expected
Absent / null / blank	Omitted; null; empty string	400, amount field error
Nonnumeric / nonfinite	abc; NaN; Infinity	400, no transaction
Excess fractional precision	0.001	400, never silently rounded
Negative / zero	-0.01; 0	400
Valid finite amount	0.01; 12.34; 9999999999.99	201, exact Decimal value
Above maximum	10000000000	400

Each slash-separated item above is a separate block. Classification order is absent, null, blank, nonnumeric, nonfinite, excessive precision, then magnitude for finite values with at most two decimal places. Thus 0.001 is classified once as excess precision. The valid block includes minimum, interior and maximum boundaries. No positive two-decimal value lies below 0.01.

Date block	Representative	Expected
Absent / null / blank	Omitted; null; empty string	400
Malformed / impossible date	25/09/2026; invalid; 2026-02-30	400
Valid calendar date	2026-09-15	201

Description length blocks are empty, interior, maximum and above maximum: 0, Food shop, 500 and 501 characters. The first three are accepted; 501 is rejected. This model covers strings, not arbitrary JSON arrays or objects.

Executable references

backend/tests/test_transactions.py: test_amount_partitions, test_missing_fields, test_invalid_dates, test_description_boundaries and test_full_crud_and_server_owned_fields. Invalid amount/reference tests assert no write as well as HTTP 400. Full block-by-block mappings and limits are in [docs/testing/ISP.md](#).

4. ISP relationships and decisions

What is checked	Cases tested	Expected result
Category selection	Missing category; nonexistent category; another user's category; own category with the wrong type; own matching category	Only an existing, owned category of the matching type is accepted.
Budget dates	Start before end; same day; start after end	The first two create a budget (201). Reversed dates are rejected (400).
Expense date	Before the budget; first day; middle; last day; after the budget	Only expenses on or between the two boundary dates count toward spending.
Login credential	Missing, malformed, expired or valid access token	The first three are rejected (401). A valid token can read the profile (200).
Record ownership	Read, edit and delete the user's own records and another user's records	Own records can be read/edited (200) or deleted (204). Records owned by others return 404 and remain unchanged.

Financial decision boundaries

Budget 100.00; spending	Remaining	Percentage	State
0.00	100.00	0.00	WITHIN_BUDGET
99.99	0.01	99.99	WITHIN_BUDGET
100.00	0.00	100.00	REACHED
100.01	-0.01	100.01	OVER_BUDGET

The inclusive-period test places expenses of 10 at the start and 20 at the end, 500 immediately outside each boundary, plus foreign-user and income records. The expected eligible total is exactly 30. An interior expense is tested separately by the budget-state cases. A single-day period has no strictly-inside block.

Each group above is tested separately. Ownership tests also combine each supported operation with own and other-user records for categories, transactions and budgets. The suite does not test every combination of every application input. Date ordering is checked only when both dates are valid; arbitrary JSON types and every possible Unicode representation are outside this input model.

References: `test_categories_budgets.py`, `test_financial_services.py`, `test_auth.py` and `test_transactions.py`. Logout adds the relation between the access-token owner and refresh-token owner; those cases discovered real authorization defects described in Section 10.

5. Backend and property testing

pytest 9.1.1 executes the suite; pytest-django 4.14.0 configures Django and a separate test database; pytest-cov 7.1.0 with coverage 7.16.1 measures statements and branches; pytest-html 4.2.0 emits readable outcomes. requirements-test.lock pins these dependencies. The application uses Django 5.2.17 and DRF 3.16.1.

Test module	Cases	Purpose
test_auth.py	29	Registration, login, expiry, rotation, logout and anonymous throttling
test_categories_budgets.py	37	CRUD, validation, ownership, references and overlapping budgets
test_database_errors_seed.py	11	PostgreSQL constraints, safe errors, seed behavior
test_filters_statistics.py	21	Filters, pagination/statistics behavior and isolation
test_financial_services.py	9	Decimal totals, inclusive periods, monthly/category aggregation
test_security_boundaries.py	13	CORS, password hashing, revalidation, no-store, SQL-payload binding
test_transactions.py	34	Amount/date/description partitions, CRUD and ownership
test_properties.py	7	Pure arithmetic examples and generated properties

Fixtures and real integrations

Fixtures create Alice, Bob, owned/foreign categories and deterministic September 2026 data. APIClient uses actual login-issued JWTs and executes serializers, views, ORM and PostgreSQL. These API/service cases are integration tests, not isolated unit tests. pytest creates a test database and applies real migrations; constraints are also exercised by direct invalid database operations.

A faster password hash is used only within the isolated test suite. A separate security test verifies the real PBKDF2 password storage and checking. PBKDF2 (Password-Based Key Derivation Function 2) uses a salt and repeated calculations to make password guessing more expensive; the original password is not stored. Other cases check safe error responses and revalidate a category that changes before saving. This is a controlled simulation, not a concurrent stress test.

Hypothesis invariants

Hypothesis 6.168.1 generates Decimal amounts from integer cents. Two properties use up to 200 deterministic examples each: remaining plus spending equals budget with correct status, and increasing spending cannot increase remaining. They complement fixed boundary cases. Generated examples are not 400 additional pytest cases; the full collected suite remains 161. The helper calculation is separated from its ORM query for genuine unit testing.

6. Frontend components and MSW

Vitest 5.0.2, React Testing Library and jsdom execute seven test files with 37 cases. Mock Service Worker (MSW) 2.15.0 intercepts requests at the HTTP boundary while real frontend services, hooks and pages run. It does not replace the service under test or prove Django behavior. `setup.js` rejects unhandled requests, resets MSW handlers, cleans rendered UI and clears `sessionStorage` between cases.

Area	Representative behavior and assertions
Components	Field labels and error associations, <code>aria-invalid/aria-describedby</code> , budget state text and visual progress cap, busy delete-dialog Escape handling.
Authentication and layout	Registration/login success and field errors; missing/expired session redirects; profile loading failure and retry; logout success clears session, failure retains it.
API service	Refresh rotation/retry, rejected refresh, malformed stored session/response, paginated reads, 204, 403, 404, 500 and network failure.
Pages	Category loading/empty/retry and CRUD; transaction form/edit/type-category filtering/pagination/deletion; budget CRUD; dashboard/statistics summaries.
Chart summaries	Exact accessible tabular/legend values and numeric data transformation; Recharts itself is stubbed in component tests.

A representative integration case

The categories endpoint first returns HTTP 500 with an internal error string. The real page must show a generic server error and omit that internal detail. Clicking Retry triggers another request, which MSW answers with an empty successful list; the page then shows its empty state. Assertions cover the state transition and request handling rather than just rendering a heading.

Coverage and scope

V8 coverage includes `src` JavaScript/JSX except `main.jsx` bootstrap and the test files. The minimums of 85% statements and 80% branches are enforced in `vitest.config.js`. Final results are 96.54% statements (363/376), 91.76% branches (234/255), 96.06% functions (122/127), and 97.13% lines (339/349). CI uploads the entire HTML coverage directory plus LCOV, preserving linked assets and per-file views.

jsdom is not a layout engine: these tests do not establish pixel geometry. Mocked chart rendering proves data/accessible summaries, not Recharts drawing. Real browser tests cover those additional boundaries. Details: [docs/testing/frontend.md](https://docs.testing.frontend.md).

7. Browser testing and CI

Playwright 1.58.0 uses Chromium 145.0.7632.6. The four UI tests comprise two isolated stylesheet fixtures with reviewed Windows screenshot baselines and two actual React registration-page tests. The latter use desktop 1440 x 900 and mobile 390 x 844 viewports to assert text, form controls, colors, explicit padding/margins, column layout and absence of horizontal scrolling.

The real-page desktop assertions include story background `rgb(18,43,57)`, white text, 40 px form padding, 32 px form top margin and 22 px heading top margin. Mobile checks include one column, 40 px 24 px padding and hidden story copy. The isolated fixture still has a narrower scope than the React page; these screenshots cover the stylesheet fixture rather than the complete live application.

One real-app E2E workflow

The browser registers a unique user; creates expense, income and temporary categories; edits/deletes the unused category; adds income 100 and expenses 20 and 5; creates/edits a budget; filters, edits and deletes one expense; verifies income 100, expenses 5 and balance 95; opens Statistics and checks monthly values, category legends and three rendered chart surfaces; deletes the budget; logs out; and registers a second user whose category list excludes the first user's records.

This workflow uses the actual React, Django and PostgreSQL stack, with no API mocks. It is one collected test containing many steps, not many independent tests. Current-month dates are generated consistently within a run; the system clock is not frozen. Unique users avoid shared-account collisions, but records remain until the disposable database is reset.

Continuous integration

Job	Execution and evidence
Backend / Ubuntu	Python 3.12, PostgreSQL service, locked dependencies, 161 tests, separate coverage gates, HTML/XML/JUnit/JSON artifacts.
Frontend / Ubuntu	Node 24, npm ci, Vite build, 37 tests and coverage; complete HTML/LCOV artifact.
UI / Windows	Chromium install, frontend install/Vite server, 4 UI checks using Windows reference images.
E2E / Ubuntu	Fresh random CI secrets, disposable Docker stack, readiness check, Chromium workflow, report upload and teardown.

All four automated GitHub Actions jobs passed. Detailed browser recordings are disabled because they can include authentication tokens. Downloadable backend reports are retained by GitHub for 30 days, and frontend/browser reports for 14 days. Selected evidence with sensitive data removed is also committed under `docs/testing/evidence/` so it remains available after those downloads expire.

8. Load testing with k6

k6 drives HTTP requests through Nginx, Gunicorn and PostgreSQL on a dedicated testing network. Twenty synthetic users have separate categories/budgets. Setup authenticates each once. Every iteration reads the summary, creates a 1.25 expense, reads it, updates it to 2.50, checks budget spending, deletes it, reads a filtered list and pauses 0.4 seconds. Assertions check exact amounts, identifiers and statuses in addition to HTTP failure metrics.

Profile	Schedule
Smoke	2 virtual users for 10 seconds.
Load	Ramp 0 to 5 over 15 s; 5 to 20 over 30 s; hold 20 for 30 s; ramp to 0 over 15 s. Graceful ramp-down 10 s.

Metric	Historical final load, 26 Sep	Combined smoke, 27 Sep
Completed iterations	2,439; zero interrupted	42; zero interrupted
HTTP requests	17,113	334
Business checks	17,073 passed; zero failed	294 passed; zero failed
HTTP failure rate	0%	0%
Request p95	22.44 ms	108.27 ms
Mean / maximum	10.24 / 149.65 ms	Not used for the acceptance gate

Both profiles met the declared thresholds: HTTP failure rate below 1%, p95 below 500 ms and checks at least 99%. The longer load profile lasted 90 seconds plus setup/graceful completion and achieved about 183.89 requests per second including setup. The earlier smoke had 44 iterations and p95 114.64 ms; it is historical, not the combined smoke above.

Interpretation and limits

p95 means 95% of observed request durations are at or below that value; it is neither the average nor the maximum nor browser rendering time. Setup requests are included. This workload is a short local load test against a small database, not a stress-to-failure, spike, soak or Internet capacity study. Login load and refresh-expiry stress are not measured. The five-minute normal access token outlasts the workload.

Recorded load environment: macOS, Docker 29.1.3 with 10 CPUs and 8,217,448,448 bytes of allocated memory, PostgreSQL 16.15, two Gunicorn workers and Nginx production frontend. Scans were run separately from performance measurements. The official k6 image is digest-pinned in `load-tests/run.py`; JSON metrics and thresholds are preserved in `evidence/k6-load-final.json`.

9. Dynamic security assessment

OWASP ZAP was run locally in a digest-pinned official Docker image against the disposable testing stack. The baseline spiders for one minute and runs passive checks. The API scan actively sends attack payloads to a generated, intentionally bounded OpenAPI surface: profile, statistics and category/transaction/budget collections and selected detail routes using GET, POST, PATCH and PUT, plus transaction filters.

DELETE and login/register/refresh/logout are excluded from active scanning to preserve fixture/session behavior; targeted backend tests cover their functional authorization. The scanner has a separate synthetic identity with a 60-minute token, while normal access tokens remain five minutes. A preflight and ZAP hook verify a successful scanner profile response without publishing tokens or bodies.

Finding / evidence	Disposition
Initial missing CSP directives, cross-origin/permissions headers and nginx version	Hardened and baseline rescanned: object-src/base-uri/form-action, COOP/COEP/CORP, Permissions-Policy, suppressed server version.
CSP inline styles: medium	Retained for current chart styling. Scripts remain same-origin without unsafe-inline. Documented residual configuration risk.
SQL injection alert: high, medium confidence	Original alert retained; reported boolean-payload case triaged as false positive using live replay and bound-query regression tests. See Section 10.
Unexpected HTML: low	SPA root, redirects and missing routes can return HTML. Unknown API-route JSON consistency remains a limitation.
Informational cache/4xx alerts	Private API no-store is deliberate; many rejected attack inputs produce 4xx. Public static caching is distinct.

The authenticated scan recorded 5,152 messages, including successful scanner identity, and an active scan state FINISHED at 100% scheduling progress with 5,076 requests. Status counts were 200:424, 201:82, 400:4,285, 404:81, 301:247, 409:2 and 0:32. Code 0 means no normal HTTP response was recorded, not a successful request.

Active scanning is bounded to five minutes globally and one minute per rule. Finished scheduling does not mean exhaustive threat coverage. Both final scans exited 2 for warnings, not a clean zero-alert result. Exit 1 denotes configured alert failures and exit 3 execution error. No unresolved confirmed high-risk finding remains from this investigation, but that is not proof that none exist.

10. Defects and retest evidence

AUTH-LOGOUT-01 Missing access authentication

Problem: Logout accepted a refresh token without requiring an authenticated access token. It returned success and revoked the refresh token. **Correction:** Logout now requires authenticated access.

Verification: A regression test confirms that an unauthenticated request is rejected with 401 and leaves the refresh token usable.

AUTH-LOGOUT-02 Refresh ownership not enforced

Problem: An authenticated user could revoke another user's refresh token if they possessed it. This demonstrated missing ownership validation, not token theft. **Correction:** Logout checks that the refresh token belongs to the authenticated account. **Verification:** A two-user regression test confirms rejection with 403 and verifies that the other user's token remains usable. The initial failures and passing retests are retained.

ZAP-40018 Investigated SQL alert

ZAP raised a possible SQL injection alert after observing different responses to two specially crafted category names. SQL injection occurs when input is interpreted as database instructions instead of ordinary data. Repeating the test with fresh names stored both as plain text; repeating either name then produced the expected duplicate-name error. Additional regression tests checked that the input remained separate from database instructions and that other users' records were unchanged.

The replay and regression tests support a false-positive classification for this reported case: the response difference was explained by duplicate-name validation, rather than demonstrated SQL execution. The original high-severity alert is retained alongside the investigation. This conclusion applies to the investigated case and is not a guarantee against every SQL injection attack.

Hardening and test-authoring failures

Private API responses were configured to prevent storage in shared caches, reducing the risk of private information being reused for another user. This was preventive hardening; no real-user data leak was demonstrated. The combined browser workflow confirmed that charts still rendered after the security-header changes.

Frontend test development exposed setup and assertion errors involving mocked responses, route parameters, numeric formatting, asynchronous loading and hidden mobile content. These were corrected and rerun. Operating-system access restrictions also interrupted some local attempts. These incidents are recorded separately from application defects; the merged-code CI runs passed.

Detailed defect log and retained before/after evidence: <docs/testing/DEFECTS.md>.

11. Reproduction setup and suites

Clone the repository and check out the tested code commit for exact reproduction. Docker Desktop must be running. Use Python 3.12 and a supported Node release: the locked jsdom declares ^22.22.2, ^24.15.0 or >=26.0.0. Node 24.21.0 was used for the recorded frontend-owner run; CI uses Node 24. Lockfiles pin packages, but mutable base image/runner tags can change environment details.

```
git clone https://github.com/erikoxha/expense-manager.git
cd expense-manager
git checkout 8907e78c8aad3c5b1581c5767f21371c3f507e2c
python3.12 -m venv .venv
. .venv/bin/activate
pip install -r backend/requirements-test.lock
python scripts/init_testing.py
docker compose -f docker-compose.testing.yml up -d --build
python scripts/run_backend.py
```

On Windows use the corresponding virtual-environment activation command and npm.cmd if PowerShell blocks npm.ps1. npm.cmd avoids changes to PowerShell execution policy. init_testing.py creates ignored random credentials. The isolated stack uses PostgreSQL on 55432, backend on 18000 and frontend on 18080. Its database is tmpfs/disposable; stopping or recreating the database container can discard its data. It is separate from the normal demo stack.

```
npm ci --prefix frontend
npm run test:coverage --prefix frontend
npm run build --prefix frontend
cd e2e
npm ci
npx playwright install chromium
npm run test:ui
```

test:ui normally starts Vite automatically. The two fixture screenshot baselines are Windows-specific; run the full visual suite on Windows or use its Windows CI job. On macOS/Linux, a new reference screenshot is not evidence of matching the reviewed Windows baseline. Reference-image changes require review of the intended appearance.

```
# From e2e/, against the disposable production build
PLAYWRIGHT_BASE_URL=http://127.0.0.1:18080 CI=1 \
  npm run test:ui -- --grep "rendered React"
PLAYWRIGHT_BASE_URL=http://127.0.0.1:18080 CI=1 \
  npm run test:e2e
```

Backend reports: reports/backend/tests.html, coverage-html/, quality-gate.json, coverage.xml and junit.xml. Frontend: frontend/coverage/. Browser: e2e/playwright-report/. JUnit XML is an output format; the backend tests use pytest, not the Java JUnit framework. Environment secrets, saved authentication state and token-bearing recordings are excluded from version control.

12. Reproducing load and security

Start the isolated testing stack as above. From the repository root, seed synthetic load/scanner users, then run the profiles separately from the scans. These tools intentionally write synthetic records and must not be repointed at real financial data or external sites.

```
docker compose -f docker-compose.testing.yml exec -T backend \
  python manage.py shell -c \
    "exec(open('/tooling/seed_testing.py').read())"
python3 load-tests/run.py smoke
python3 load-tests/run.py load
python3 security/run_scan.py baseline
python3 security/prepare_scan.py
python3 security/run_scan.py api
python3 security/reproduce_category_alert.py
```

The seed helper checks the dedicated expense_testing database name, prepares 20 separate users and scanner-owned fixtures, and is repeatable. k6 authenticates through the normal login API; its runner uses the dedicated Docker network. security/prepare_scan.py stores the scanner credential only in ignored .zap.env and writes the bounded OpenAPI definition and sanitized preflight.

ZAP can mutate scanner data. Recreate/seed disposable fixtures when needed. Scan interpretation combines the exit status, authenticated identity evidence and alerts. A warning exit is not automatically a failure of the business application and is never silently called clean. The final runner checks for authentication evidence; its added missing-evidence failure guard has syntax review but was not separately exercised with a deliberately failed authentication run.

Where to inspect evidence

Repository location	Contents
docs/testing/evidence/	Backend gates/outcomes; original logout failures; historical k6 summaries; original and final ZAP alerts; SQL replay and authentication evidence.
docs/testing/evidence/combined/	Combined backend/frontend/UI/E2E outcome logs, k6 smoke and exact parent/tree identifiers.
docs/testing/ISP.md	Full partition tables, concrete representatives, expected results and exact test function mappings.
GitHub Actions artifacts	HTML coverage/tests, XML/LCOV, browser reports; downloads expire after 14/30 days; selected evidence is retained in the repository.

The historical evidence manifest identifies then-uncommitted files by SHA256 and a base commit. It is not a claim that the old base commit contained all tests. The source-version link and CI runs in the final section identify the integrated code used for verification. Raw local reports are ignored; reviewed sanitized evidence is versioned.

13. Final evidence and conclusions

Final merged-main check	Verified result
Backend CI	161 passed in 6.83 s; statements 98.94% (373/377), branches 100% (60/60). Four WSGI bootstrap statements unexecuted.
Frontend CI	37 passed; statements 96.54%, branches 91.76%. Vite production build passed.
Browser UI CI	4 passed on Windows, including 2 fixture and 2 real-page checks.
Real-app E2E CI	1 passed on Ubuntu; final merged API, database and frontend workflow.

Final test runs

[Backend tests and coverage](#)
[Frontend, browser UI and end-to-end tests](#)
[Tested application source version](#)

Backend and frontend local integration checks also passed before merge. The 90-second load and bounded ZAP measurements are historical runs, not reruns on the final merge. The frontend testing contribution changed no product components or production dependency versions; combined tests rechecked the affected logout and chart behavior. Neither coverage nor a scanner establishes complete correctness or security.

Limitations and future improvements

The project does not test every execution path, the independent effect of every combined condition, all input combinations, sustained capacity, multi-browser compatibility or a full penetration test. Coverage excludes specified bootstrap/migration code. UI fixtures have limited scope and snapshots depend on OS/browser. E2E uses unique synthetic users but no account-deletion API. Authentication uses sessionStorage, so XSS could expose tokens; short-lived JWTs remain valid until expiry. Local-cache throttling is not a distributed production defense. Inline-style CSP remains a medium finding and unknown API routes may return HTML.

Future work could add long-duration/spike load, broader browser coverage, mutation testing, richer API error-format checks, and a production authentication/cache threat-model review. Those are proposals, not completed results.

Primary tool references

[pytest and fixtures](#); [pytest-django database testing](#); [coverage.py branch measurement](#); [Hypothesis](#); [Vitest coverage](#); [React Testing Library](#); [MSW](#); [Playwright assertions](#); [k6 thresholds](#); [ZAP API scan](#); [ZAP scan hooks](#); [Django password storage](#).

The repository contains detailed execution instructions, test mappings and retained evidence. Earlier measurements remain labeled historical; the final results table summarizes the verified merged application.